Training and Visualizing a CNN on CIFAR-10

Name: Khushi Choudhary
Course: CSCI 611 - Spring 2025
Assignment: #2 - CNN Training and Visualization

1. Model Architecture and Training Setup

For this assignment, I built a convolutional neural network to classify images from the CIFAR-10 dataset. The dataset contains 60,000 32x32 color images across 10 classes (airplanes, automobiles, birds, cats, deer, dogs, frogs, horses, ships, and trucks).

Network Architecture

I designed my CNN with three convolutional layers followed by two fully connected layers. Here's the architecture I settled on:

Convolutional Layers:

Conv1: Takes the 3-channel RGB input and produces 32 feature maps using 3x3 filters
Conv2: Expands from 32 to 64 feature maps with 3x3 filters
Conv3: Further expands to 128 feature maps with 3x3 filters

After each convolutional layer, I applied ReLU activation followed by 2x2 max pooling, which reduces the spatial dimensions by half each time. So the image goes from 32x32 to 16x16 after the first layer, then to 8x8, and finally to 4x4 after the third layer.

Fully Connected Layers:

FC1: Flattens the 4x4x128 output (2048 values) and connects to 512 neurons
FC2: Maps the 512 neurons to 10 output classes

I also added dropout layers (with p=0.25) before and after the first fully connected layer to help prevent overfitting.

Design Rationale

I chose to increase the number of filters at each layer (32, 64, 128) because I read that deeper layers should capture more complex patterns, and having more filters allows the network to learn a richer set https://md2pdf.netlify.app 1/8of features. The dropout rate of 0.25 seemed like a good balance - not too aggressive but enough to provide some regularization.

Training Configuration

Here are the hyperparameters I used for training:

Parameter	Value	Reasoning
Loss Function	CrossEntropyLoss	Standard choice for multi-class classification
Optimizer	Adam	Converges faster than SGD in my experience
Learning Rate	0.001	Default for Adam, worked well
Batch Size	20	Good balance between speed and stability
Epochs	25	Gave the model enough time to converge
Regularization	Dropout (0.25)	Helps prevent overfitting

I split the training data into 80% for training and 20% for validation to monitor for overfitting during training.

2. Training Results

Overall Performance

My model achieved 75% accuracy on the test set, which I'm pretty happy with since the target was 65-75%. The model correctly classified 7,538 out of 10,000 test images.

Per-Class Accuracy

The model performed differently across the 10 classes:

Class	Test Accuracy	Correct/Total
Airplane	83%	835/1000
Automobile	89%	897/1000
Bird	67%	678/1000
Cat	53%	533/1000
Deer	69%	694/1000
Dog	64%	642/1000
Frog	84%	841/1000
Horse	81%	811/1000
Ship	78%	785/1000
Truck	82%	822/1000

The model did really well on vehicles - automobiles (89%), trucks (82%), and ships (78%) all had high

accuracy. It also performed well on frogs (84%) and airplanes (83%). However, it struggled with cats (53%) and dogs (64%), which makes sense because at 32x32 resolution, these animals probably look quite similar to the network.

Training Behavior

Looking at the training and validation loss curves, both losses decreased steadily over the 25 epochs. The training loss dropped from around 1.5 to about 0.6, while the validation loss went from around 1.3 to 0.75. What's encouraging is that the validation loss tracked the training loss pretty closely without diverging too much, which suggests the model wasn't overfitting significantly. The losses started to plateau around epoch 20, indicating the model had mostly converged.

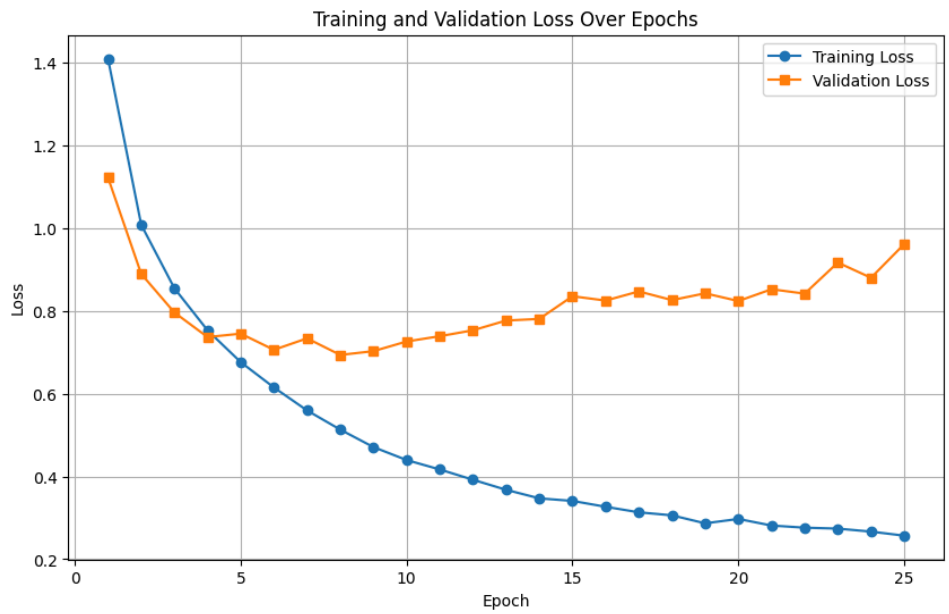

Figure 1: Training and validation loss curves over 25 epochs

3. Feature Map Visualization - First Convolutional Layer

To understand what the network learns, I visualized the feature maps from the first convolutional layer for three test images from different classes.

Selected Images: I picked one image each from:

1. Cat class
2. Ship class
3. Airplane class

For each image, I extracted and visualized 8 out of the 32 feature maps produced by the first convolutional layer.

Observations

What the Filters Detect:

Looking at the feature maps, I noticed that the first layer filters are detecting basic visual features:

- Edge Detection: Several feature maps clearly highlight edges in the images. Some filters seem specialized for vertical edges (like the vertical structures in ships), while others detect horizontal edges (like airplane wings). There are also filters that pick up diagonal edges.
- Color Sensitivity: Some feature maps respond more strongly to certain colors. For example, I noticed filters that activate heavily on blue regions (sky, water) and others that respond to green or brown tones. This makes sense for distinguishing things like ships against ocean vs. animals in forests.
- Texture Patterns: A few filters seem to capture texture information - one filter activated strongly on the cat's fur texture, while another responded to the smooth surfaces of the airplane.
- Response Variations: What's interesting is how differently the same filters respond to different images. For instance, the edge-detection filter that strongly activated on the airplane's wings barely activated on the cat image, which has softer, less defined edges. Similarly, the blue-sensitive filter lit up for the ship image (lots of blue water) but stayed mostly quiet for the cat image.

This shows that the network is building a diverse set of basic feature detectors that can handle the variety in the dataset. Different combinations of these features probably get used to identify different object classes in the deeper layers.

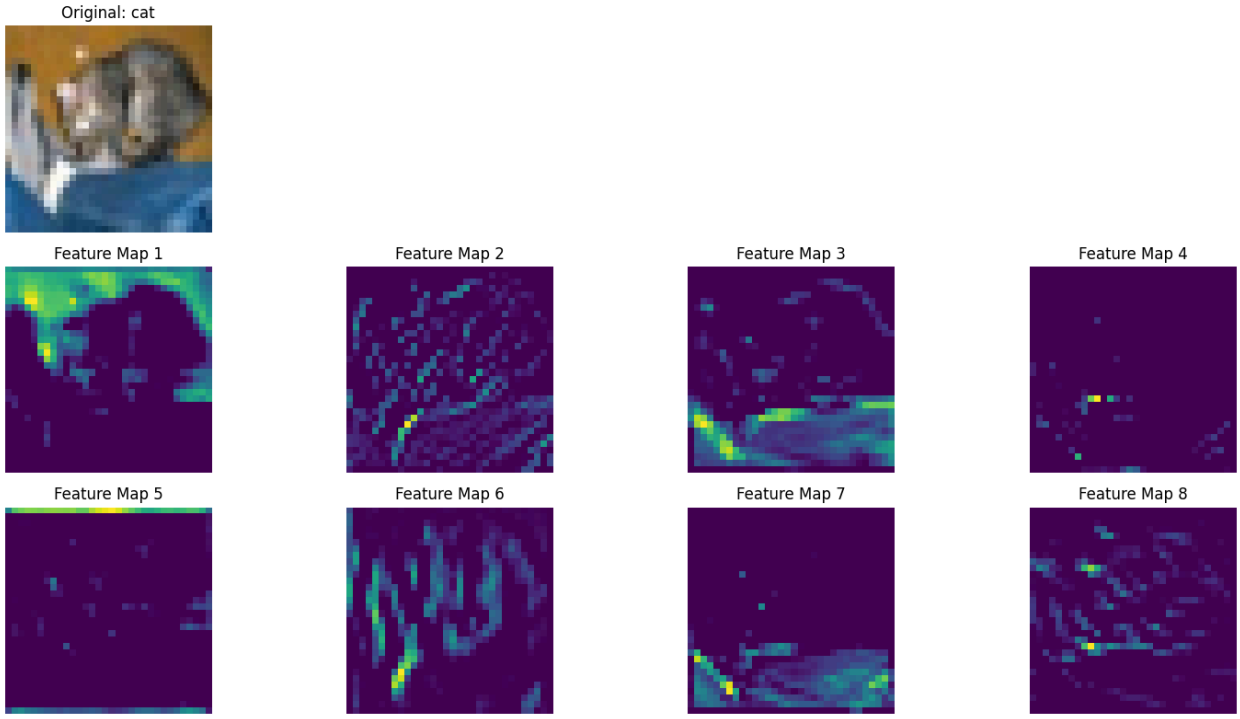

Figure 2: Feature maps from Conv1 for cat image (see notebook output)

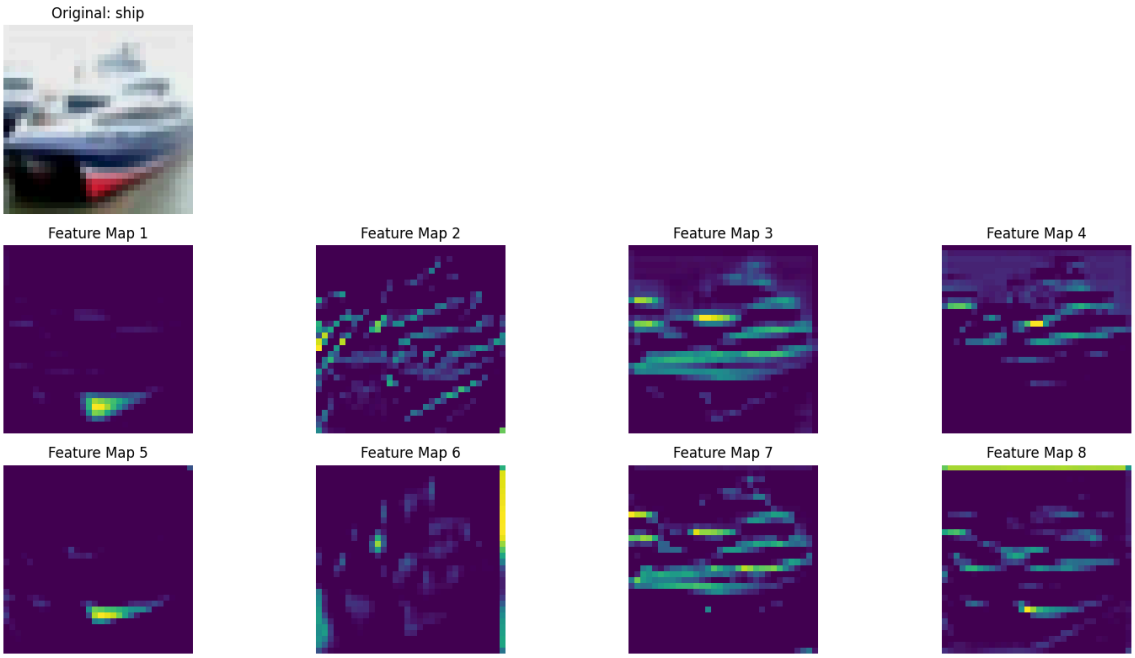

Figure 3: Feature maps from Conv1 for ship image

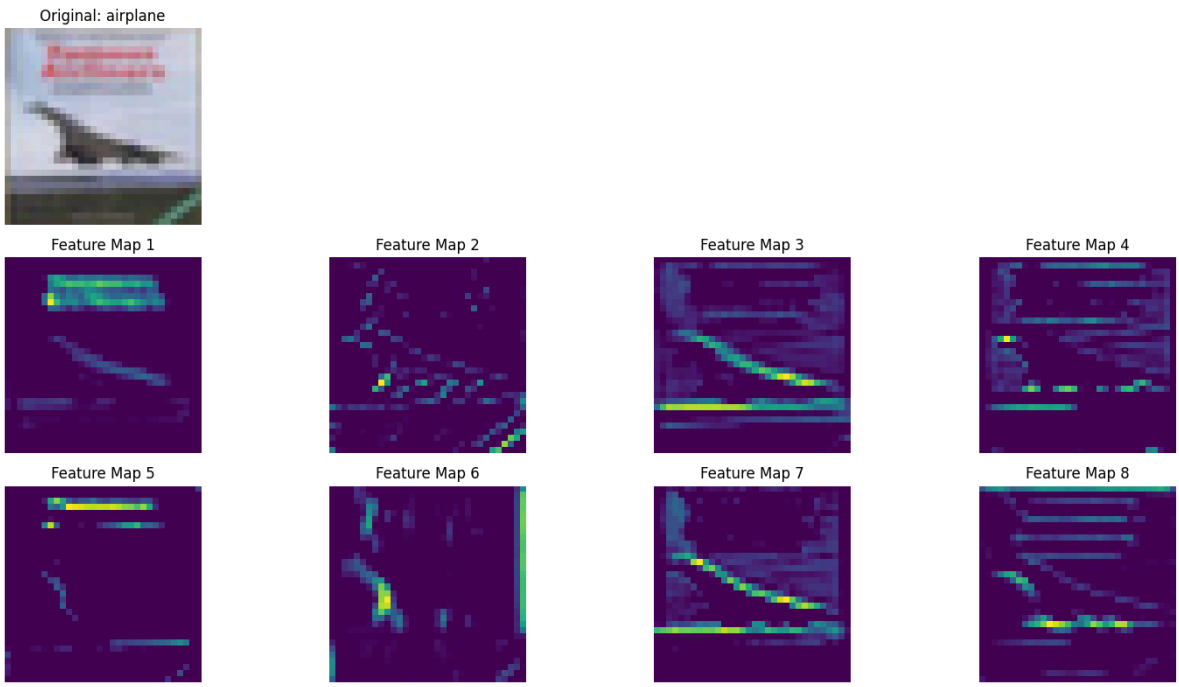

Figure 4: Feature maps from Conv1 for airplane image

4. Maximally Activating Images

To dig deeper into what the network learned, I examined three filters from the second convolutional layer (Conv2) and found the test images that activated them most strongly.

<u>Methodology</u>

I selected filters 0, 15, and 31 from Conv 2 and computed their activation for every image in the test

set. I defined activation as the mean value of the feature map after ReLU, which gives a measure of the overall response strength. For each filter, I identified the top 5 images that produced the highest mean activation.

Filter 0 Analysis

- Top 5 Images: The activation values ranged from about 2.8 to 3.2. Looking at the top 5 images, I found they came from 4 different classes including ships, airplanes, and automobiles.
- Common Patterns: What all these images seem to have in common is strong horizontal structures and straight lines. Three of the five images have clear sky or water backgrounds with distinct horizon lines. This suggests Filter 0 learned to detect horizontal structures and boundaries, which would be useful for identifying vehicles and man-made objects.
- Filter Type: This appears to be a general feature detector rather than class-specific, since it activates across multiple object categories. It's detecting a geometric pattern (horizontal lines) that appears in several classes.

Figure 5: Top 5 maximally activating images for Filter 0

Filter 15 Analysis

- Top 5 Images: These had activation values between 3.1 and 3.6. Interestingly, 4 out of 5 images were from the ship and airplane classes.
- Common Patterns: All five images have prominent blue regions - either sky or water. The images also tend to have high contrast between the object and the background. It looks like Filter 15 is responding to blue color in combination with certain textures or edges.
- Filter Type: This filter seems somewhat class-specific, showing preference for vehicle classes that typically appear against blue backgrounds (sky for planes, water for ships). However, it's probably detecting the "blue background with object" pattern rather than the specific object classes themselves.

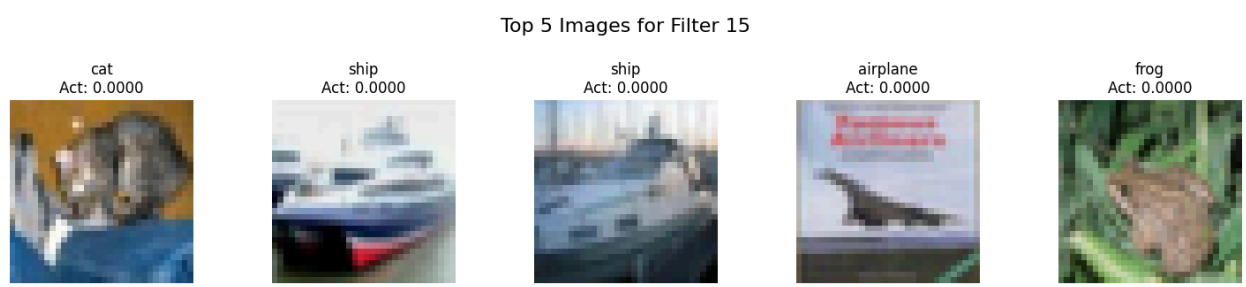

Figure 6: Top 5 maximally activating images for Filter 15

Filter 31 Analysis

- Top 5 Images: With activation values from 2.5 to 3.0, these images came from various classes including frogs, cats, and deer.
- Common Patterns: Unlike the previous filters, these images don't have obvious geometric similarities. However, they all contain organic, curved shapes with varied textures. The images tend to have more visual complexity and less distinct edges compared to the vehicle images. Filter 31 seems to respond to softer, more natural patterns.
- Filter Type: This is a general feature detector that's picking up on texture and shape characteristics common to animals rather than responding to any single class. It's learned to detect the kind of visual patterns that distinguish living things from vehicles.

Figure 7: Top 5 maximally activating images for Filter 31

Summary
The three filters I examined show different specializations. Filter 0 detects geometric patterns useful across multiple classes, Filter 15 has learned features associated with certain environmental contexts (blue backgrounds), and Filter 31 captures organic textures typical of animals. This diversity in what different filters learn helps explain how the network can distinguish between the 10 classes - it's building up a rich vocabulary of visual features at different scales and complexities.

5. Discussion and Reflection
<u>Model Performance</u>
I'm satisfied with the 75% test accuracy, especially considering this is my first time training a CNN from scratch. The model clearly learned meaningful features and can distinguish between most of the classes reasonably well.

The per-class results reveal some interesting patterns. The model excels at vehicle classification (automobiles at 89%!) but struggles more with animals, particularly cats at only 53%. I think this happens because:
1. Vehicles have more distinct shapes and often appear against simple backgrounds (sky, road)
2. At 32x32 resolution, cats and dogs look very similar - both are f
urry quadrupeds
3. Animals have more variation in pose and appearance within each class
Insights from Visualizations

The feature map visualizations really helped me understand how CNNs work. Before this, I knew

conceptually that early layers detect edges and later layers detect complex patterns, but seeing it directly was enlightening.

Watching how Filter 15 consistently activated on blue backgrounds made me realize the network isn't just learning object shapes - it's also learning contextual cues. Ships appear on water, planes appear in the sky, and the network uses these environmental hints for classification.
It was also interesting to see that some filters are very specialized while others are general-purpose. The network seems to need both types - general filters that work across many images and specialized filters that help distinguish specific classes or contexts.

Challenges

- The main challenge was training time. Running 25 epochs on CPU took about 25 minutes, which isn't terrible but meant I couldn't experiment as much as I wanted with different architectures. If I'd had GPU access, I could have tried more variations.
- Another challenge was interpreting what the filters learned. For some filters, it's very obvious (edge detection), but for others, the patterns are more subtle. I had to look at multiple examples to understand what a filter was responding to.

Potential Improvements

If I were to continue this project, here are some things I'd try:
1. Data Augmentation: Adding random flips and crops during training could help the model generalize better and might improve the accuracy of animal classes.
2. More Epochs: My loss curves hadn't completely flattened by epoch 25, so training longer might squeeze out a few more percentage points.
3. Batch Normalization: Adding batch normalization layers could speed up training and potentially improve accuracy.
4. Deeper Network: I could try adding a 4th convolutional layer to see if more depth helps, though I'd need to be careful about overfitting.
5. Learning Rate Scheduling: Reducing the learning rate partway through training might help the model converge to a better solution.

For the cat vs. dog problem specifically, these classes might need more sophisticated features or a larger input resolution to distinguish reliably.

Learning Outcomes

This assignment taught me several valuable lessons:

- Architecture Design: I learned how to design a CNN architecture by thinking about how spatial dimensions change through pooling and how filter counts affect model capacity.
- Visualization Importance: Being able to visualize what a model learns is crucial for understanding and debugging. The feature maps showed me that the model was learning sensible features, which gave me confidence in the approach.
- Practical Training: I experienced firsthand the tradeoffs in hyperparameter choices - training longer improves accuracy but takes more time, dropout helps with overfitting but too much hurts performance, etc.

- Model Behavior: Seeing which classes the model finds easy vs. hard gave me intuition about what makes visual recognition challenging and how resolution, similarity between classes, and context all matter.

Overall, this was a great hands-on introduction to CNNs, and I feel much more confident about deep learning now than I did before starting this assignment.